

AN PROXY SERVER

Proxy Server

1. Proxy Server Overview

A **proxy server** acts as an intermediary between a client and a destination server.

There are two main types:

- **Forward Proxy** → represents the client
- **Reverse Proxy** → represents the server

Forward Proxy:

Client → Proxy → Internet

Reverse Proxy:

Client → Proxy → Backend Server

2. Forward Proxy — Squid

2.1 What is Squid?

Squid is an open-source forward proxy and caching proxy. It can control, filter, log and cache web traffic.

Main Functions

- Forward proxy
 - Website access control
 - Website blocking
 - Caching
 - Traffic logging
-

2.2 Lab Network Configuration

Machine	IP Address	Role
Squid Server	172.16.0.1	Forward Proxy
Windows Client	172.16.0.3	Proxy Client

The Squid server should have connectivity to the **client network and Internet**.

Example Squid internal interface:

```
IP Address : 172.16.0.1
Subnet      : 255.255.255.0
```

Check:

```
ip a
```

Test:

```
ping 172.16.0.3
ping 8.8.8.8
```

2.3 Install Squid

```
sudo apt update  
sudo apt install squid -y
```

Check:

```
squid --version
```

Configuration file:

```
/etc/squid/squid.conf
```

2.4 Backup Configuration

```
sudo mkdir -p /home/star/backups  
sudo cp /etc/squid/squid.conf /home/star/backups/squid.conf
```

Edit:

```
sudo nano /etc/squid/squid.conf
```

2.5 Squid Configuration

```
http_port 3128

acl localnet src 172.16.0.0/24
acl blocked_sites dstdomain "/etc/squid/web.txt"

http_access deny blocked_sites
http_access allow localnet
http_access deny all

cache_dir ufs /var/spool/squid 100 16 256
maximum_object_size 4096 KB
```

Important Directives

Directive	Purpose
http_port 3128	Squid listens on port 3128
acl	Defines an access-control rule
localnet	Defines allowed client network
dstdomain	Matches destination domains
http_access deny	Denies matching requests
http_access allow	Allows matching requests
cache_dir	Defines disk cache
maximum_object_size	Limits cached object size

Rule order is important: the blocking rule must come before the general allow rule.

2.6 Website Blocking

Create the block list:

```
sudo nano /etc/squid/web.txt
```

Example:

```
.chatgpt.com
.instagram.com
.youtube.com
.facebook.com
```

Request Flow

1. Client sends request to Squid.
 2. Squid checks the ACL.
 3. If the domain matches `blocked_sites` → **DENY**.
 4. Otherwise → request is forwarded to the Internet.
-

2.7 Apply Configuration

Check:

```
sudo squid -k parse
```

Restart and enable:

```
sudo systemctl restart squid  
sudo systemctl enable squid
```

Check service and port:

```
systemctl status squid  
sudo ss -lntp | grep 3128
```

2.8 Configure Windows Client

Network:

```
IP Address      : 172.16.0.3
Subnet Mask     : 255.255.255.0
Gateway        : 172.16.0.1
DNS             : 172.16.0.1
Alternate DNS   : 8.8.8.8
```

Windows **Manual Proxy**:

```
Proxy : 172.16.0.1
Port  : 3128
```

2.9 Testing

Allowed website

```
Client → Squid → Internet → ALLOW
```

Blocked website

```
Client → Squid → ACL → DENY
```

View requests:

```
sudo tail -f /var/log/squid/access.log
```

2.10 Caching

Squid can store suitable web content locally.

```
cache_dir ufs /var/spool/squid 100 16 256
maximum_object_size 4096 KB
```

Flow:

```
First request → Internet → Squid Cache
Later request → Squid Cache (if cacheable)
```

Not all modern web/HTTPS content can be cached.

2.11 Failure Testing

Stop Squid

```
sudo systemctl stop squid
```

Try accessing the Internet through the Windows proxy.

Restore:

```
sudo systemctl start squid
```

Test incorrect proxy port

Temporarily set the Windows proxy to:

```
Proxy : 172.16.0.1
Port  : 3000
```

Internet access through the proxy should fail.

Restore:

```
Proxy : 172.16.0.1
Port  : 3128
```

2.12 Troubleshooting

Problem	Check
Squid won't start	<code>sudo squid -k parse</code>
Client cannot connect	Check IP/network
Proxy refused	<code>ss -lntp grep 3128</code>
Website not blocked	Check <code>web.txt</code> , ACL and rule order
All websites blocked	Check <code>localnet</code> ACL
Internet unavailable	Check Squid server's Internet connection
Need to see requests	<code>tail -f /var/log/squid/access.log</code>

2.13 Real-World Scenario

A company sends employee Internet traffic through a central Squid server.

Employees → Squid → Internet

The administrator can control access, block websites, log traffic and use caching where applicable.

2.14 Other Forward Proxy Tools

- **Tinyproxy** — lightweight proxy
 - **Privoxy** — filtering/privacy-focused proxy
 - **CCProxy** — proxy server for Windows environments
 - **Tor** — privacy/anonymity network that can provide proxy-style access
-

2.15 Squid Viva

Q. What is Squid?

An open-source forward and caching proxy.

Q. Default Squid port?

3128.

Q. What is an ACL?

A rule used to match and control traffic.

Q. What does `dstdomain` do?

Matches the destination domain.

Q. Why is `http_access` order important?

Rules are evaluated in order.

Q. How do you check Squid configuration?

```
sudo squid -k parse
```

3. Reverse Proxy — HAProxy

3.1 HAProxy Overview

HAProxy is a reverse proxy and load balancer. It receives client requests and distributes them among backend servers.

Main Functions

- Reverse proxy
 - Load balancing
 - Health checking
 - High availability
-

3.2 Lab Network Configuration

Machine	IP Address	Role
HAProxy Server	172.16.0.1	Reverse Proxy
Web Server 1	172.16.0.2	Apache
Web Server 2	172.16.0.3	Apache
Client	172.16.0.4	Testing

All machines should be on the same network.

Example HAProxy network configuration:

```
allow-hotplug enp0s9
iface enp0s9 inet static
    address 172.16.0.1
    subnet 255.255.0.0
```

Check:

```
ip a
```

Test:

```
ping 172.16.0.2
ping 172.16.0.3
```

3.3 Configure Web Servers

On both web servers:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
```

Use different `index.html` files:

Web Server 1

```
<h1>Web Server 1</h1>
```

Web Server 2

```
<h1>Web Server 2</h1>
```

Verify from HAProxy server:

```
curl http://172.16.0.2
curl http://172.16.0.3
```

This verifies network connectivity, Apache and port 80 together.

3.4 Install HAProxy

```
sudo apt update
sudo apt install haproxy -y
```

Backup:

```
sudo mkdir -p /home/star/backups
sudo cp /etc/haproxy/haproxy.cfg /home/star/backups/haproxy.cfg
```

Edit:

```
sudo nano /etc/haproxy/haproxy.cfg
```

3.5 HAProxy Configuration

```
frontend http_front
  bind *:80
  default_backend http_back

backend http_back
  balance roundrobin
  server s1 172.16.0.2:80 check
  server s2 172.16.0.3:80 check
```

Request Flow

1. Client sends request to HAProxy on port 80.
2. `frontend` receives it.
3. `default_backend` sends it to `http_back`.
4. `roundrobin` selects a backend server.
5. Apache processes the request.
6. Response returns through HAProxy to the client.

Important Directives

Directive	Purpose
<code>frontend</code>	Receives client requests
<code>bind *:80</code>	Listens on port 80
<code>default_backend</code>	Selects backend
<code>backend</code>	Defines backend servers
<code>roundrobin</code>	Distributes requests
<code>server</code>	Defines backend server
<code>check</code>	Checks server availability

3.6 Apply and Test

Validate:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Restart and enable:

```
sudo systemctl restart haproxy
sudo systemctl enable haproxy
```

Check service and port:

```
systemctl status haproxy
sudo ss -lntp | grep :80
```

From client:

```
curl http://172.16.0.1
```

Repeat requests:

```
Web Server 1 → Web Server 2 → Web Server 1 → Web Server 2
```

This demonstrates **round-robin load balancing**.

3.7 Failure Testing

Stop Apache on Web Server 1:

```
sudo systemctl stop apache2
```

Test:

```
curl http://172.16.0.1
```

HAProxy detects the unavailable server through `check` and continues using Web Server 2.

Restore:

```
sudo systemctl start apache2
```

Test again.

3.8 Troubleshooting

Problem	Check
HAProxy won't start	<code>haproxy -c -f /etc/haproxy/haproxy.cfg</code>
Client cannot reach proxy	Check IP/network
Backend unavailable	<code>curl http://172.16.0.2 / .3</code>
Port 80 problem	<code>ss -lntp grep :80</code>
Only one server receives traffic	Check Apache, IPs and <code>roundrobin</code>
Backend marked unavailable	Check Apache, port 80 and <code>check</code>
Wrong backend	Check <code>s1</code> and <code>s2</code> IPs

3.9 Basic Security & Maintenance

For production:

- Use HTTPS/TLS where required.
- Restrict direct access to backend servers.
- Keep HAProxy and backend software updated.
- Monitor service and backend availability.
- Keep configuration backups.

3.10 Real-World Scenario

A company has two web servers running the same application. Users connect to HAProxy, which distributes requests between the servers and avoids an unavailable backend.

3.11 HAProxy Viva

Q. What is HAProxy?

A reverse proxy and load balancer.

Q. What does `roundrobin` do?

Distributes requests sequentially between backend servers.

Q. What does `check` do?

Checks backend server availability.

Q. What is a frontend?

The client-facing section that receives requests.

Q. What is a backend?

The section containing backend servers.

4. Reverse Proxy — Nginx

4.1 Nginx Overview

Nginx is a web server that can also work as a reverse proxy.

Main uses:

- Reverse proxy
 - Load balancing
 - SSL/TLS termination
 - Caching
-

4.2 Lab Network Configuration

Machine	IP Address	Role
Nginx Server	172.16.0.1	Reverse Proxy
Web Server	172.16.0.2	Apache
Client	172.16.0.4	Testing

Example:

```
allow-hotplug enp0s9
iface enp0s9 inet static
    address 172.16.0.1
    subnet 255.255.0.0
```

Check:

```
ip a
```

4.3 Configure Backend

Install Apache:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable --now apache2
```

Create a test page:

```
<h1>Backend Web Server</h1>
```

Test:

```
curl http://172.16.0.2
```

4.4 Install Nginx

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable --now nginx
```

Check:

```
systemctl status nginx
```

4.5 Nginx Reverse Proxy Configuration

Edit:

```
sudo nano /etc/nginx/sites-available/default
```

Use:

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass http://172.16.0.2:80;  
    }  
}
```

Important Directives

Directive	Purpose
listen 80	Listens on port 80
location /	Handles incoming requests
proxy_pass	Forwards requests to backend

Request Flow

1. Client sends request to Nginx.
 2. Nginx receives it on port 80.
 3. location / matches the request.
 4. proxy_pass forwards it to Apache.
 5. Response returns through Nginx.
-

4.6 Apply and Test

Validate:

```
sudo nginx -t
```

Restart:

```
sudo systemctl restart nginx
```

Check service and port:

```
systemctl status nginx  
sudo ss -lntp | grep :80
```

From client:

```
curl http://172.16.0.1
```

Expected:

```
Backend Web Server
```

4.7 Failure Testing

Stop Apache:

```
sudo systemctl stop apache2
```

Test:

```
curl http://172.16.0.1
```

The request will fail because the backend is unavailable.

Restore:

```
sudo systemctl start apache2
```

Test again.

4.8 Troubleshooting

Problem	Check
Nginx won't start	<code>systemctl status nginx</code>
Configuration error	<code>sudo nginx -t</code>
Backend unavailable	<code>curl http://172.16.0.2</code>
Port 80 conflict	<code>ss -lntp grep :80</code>
Wrong backend	Check <code>proxy_pass</code> IP
Client cannot connect	Check network/IP

4.9 Basic Security & Maintenance

For production:

- Use HTTPS/TLS where required.
- Keep Nginx and backend software updated.
- Restrict direct backend access.
- Monitor logs and service availability.
- Keep configuration backups.

4.10 Real-World Scenario

Nginx can provide a single entry point for users while the actual application runs on a backend server.

Users → Nginx → Application Server

4.11 Nginx Viva

Q. What is Nginx used for here?

As a reverse proxy.

Q. What does `proxy_pass` do?

It forwards requests to the backend server.

Q. How do you validate Nginx configuration?

`sudo nginx -t`

Q. What happens if the backend is down?

Nginx cannot obtain a response from the backend.

5. Forward Proxy vs Reverse Proxy

Feature	Forward Proxy	Reverse Proxy
Represents	Client	Server
Example	Squid	HAProxy, Nginx
Main direction	Client → Proxy → Internet	Client → Proxy → Backend
Main use	Access control/filtering	Load balancing/reverse proxying
Typical client awareness	Client is configured to use proxy	Client normally accesses proxy/service

Simple Difference

Forward:

Client → Squid → Internet

Reverse:

Client → HAProxy/Nginx → Web Server

6. Final Practical Summary

Squid — Forward Proxy

Client



Squid :3128



Internet

Main concepts: ACL, website blocking, caching, logging.



Squid

Squid Proxy (Forward Proxy)

Controls, filters, caches and optimizes client internet access

- ✓ Access Control
- ✓ Web Filtering
- ✓ Caching

- ✓ Bandwidth Management
- ✓ Logging & Monitoring
- ✓ Centralized Internet Access

What is Squid?

Squid is an open-source web proxy and caching server that acts as a forward proxy. It sits between clients and the internet, receives client requests, and forwards them to the internet on their behalf. It can also cache content, control access, filter websites and log traffic.

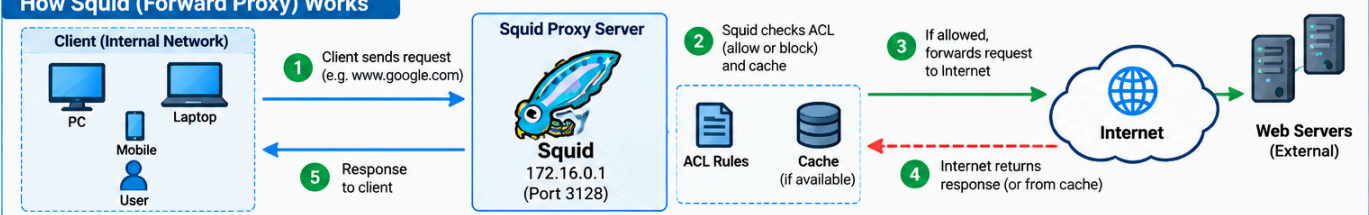
Main Uses

- Forward proxy (client-side proxy)
- Web access control and filtering
- Website blocking (ACL)
- Caching (improves performance)
- Bandwidth management (with additional configuration)
- Logging and monitoring
- Centralized internet access for an organization
- Authentication / user-based access control (optional)
- Privacy / hides client IP from destination sites

Key Information

Default Port	: 3128 (HTTP)
Configuration File	: /etc/squid/squid.conf
Cache Directory	: /var/spool/squid
Access Log	: /var/log/squid/access.log
Main Service	: squid
Client Configuration	: Required (manual or via PAC)
Supported Protocols	: HTTP, HTTPS (CONNECT)
Platforms	: Linux, Windows and others

How Squid (Forward Proxy) Works



```
graph LR
    subgraph Client [Client Internal Network]
        PC[PC]
        Laptop[Laptop]
        Mobile[Mobile]
        User[User]
    end
    subgraph Squid [Squid Proxy Server]
        Squid[Squid 172.16.0.1 Port 3128]
    end
    subgraph Internet [Internet]
        WebServers[Web Servers External]
    end
    Client -- "1 Client sends request (e.g. www.google.com)" --> Squid
    Squid -- "2 Squid checks ACL (allow or block) and cache" --> Internet
    Internet -- "3 If allowed, forwards request to Internet" --> WebServers
    WebServers -- "4 Internet returns response (or from cache)" --> Squid
    Squid -- "5 Response to client" --> Client
```

Squid Configuration (Example)

```
# Listen on port 3128
http_port 3128

# Define local network (allowed clients)
acl localnet src 172.16.0.0/24

# Define blocked websites list
acl blocked_sites dstdomain "/etc/squid/web.txt"

# Access rules (order matters)
http_access deny blocked_sites
http_access allow localnet
http_access deny all

# Cache settings
cache_dir ufs /var/spool/squid 100 16 256
maximum_object_size 4096 KB
```

Website Block List (/etc/squid/web.txt)

```
.youtube.com
.instagram.com
.facebook.com
.example-blocked.com

# Add one domain per line
```

Client Configuration (Windows Example)

1. Open Settings → Network & Internet → Proxy
2. Select "Use a proxy server (Manual)"
3. Address: 172.16.0.1
4. Port: 3128
5. Save and apply

Logs

View live access logs:

```
sudo tail -f /var/log/squid/access.log
```

Example log entry:

```
172.16.0.3 TCP_MISS/200 5243 GET
http://www.google.com/ - HIER_DIRECT/
142.250.72.68 text/html
```

Start / Restart Service

```
sudo systemctl restart squid
sudo systemctl enable squid
sudo systemctl status squid
sudo squid -k parse # Check configuration
```

Failure Testing

1. Stop Squid and test internet access
2. Try accessing a website (should fail)
3. Start Squid again
4. Test again (should work)

Common Issues & Troubleshooting

Squid not starting	Check configuration (squid -k parse)
Client cannot connect	Check IP, proxy settings and port
Website not blocked	Check ACL and rule order
All websites blocked	Check localnet ACL
No internet access	Check Squid server's internet connectivity
Need to see requests	Check /var/log/squid/access.log

Summary

Squid is a powerful forward proxy server that helps organizations control and monitor internet access, block unwanted content and improve performance through caching. It is widely used in enterprise networks and can be extended with authentication, bandwidth management and advanced filtering.

"Control today,
a safer internet tomorrow."

HAProxy — Reverse Proxy

Client



HAProxy :80



Web Server 1 / Web Server 2

Main concepts: frontend, backend, round-robin, health check.



HAProxy (Reverse Proxy / Load Balancer)

Distributes client requests to multiple backend servers

*Reliable
Flexible
High Performance*

What is HAProxy?

HAProxy is a high-performance, open-source reverse proxy and load balancer. It sits in front of backend servers, receives client requests (usually from the internet), and forwards them to one of the backend servers based on a configured algorithm (e.g., round-robin). It also performs health checks and can hide the real server IPs from clients.

Main Uses

- Reverse proxy (HTTP/HTTPS)
- Load balancing (round-robin, leastconn, source, etc.)
- High availability
- Backend health checking
- SSL/TLS termination (port 443)
- Hiding real server IP addresses
- Access control and authentication (optional)
- Connection management
- Caching (with additional modules)
- Application traffic management

Key Information

HTTP Port	: 80
HTTPS Port	: 443
Configuration File	: /etc/haproxy/haproxy.cfg
Main Service	: haproxy
Common Algorithms	: roundrobin, leastconn, source, etc.
Health Check	: Yes (check)
Client Configuration	: Not required (works transparently)
Statistics (optional)	: http://<IP>:8404 (if enabled)
Platforms	: Linux (widely used)

How HAProxy (Reverse Proxy) Works



```
graph LR
    subgraph Clients [Clients (Users)]
        PC[PC]
        Laptop[Laptop]
        Mobile[Mobile]
    end
    subgraph HAProxy [HAProxy Server (Reverse Proxy)]
        HAProxy[HAProxy]
    end
    subgraph Backend [Backend Web Servers]
        WS1[Web Server 1  
172.16.0.2  
(Apache / Application)]
        WS2[Web Server 2  
172.16.0.3  
(Apache / Application)]
    end
    Clients -- "1. Client sends request (HTTP/HTTPS) e.g. http://172.16.0.1" --> HAProxy
    HAProxy -- "2. Selects a healthy backend server (based on algorithm, e.g. round-robin)" --> WS1
    HAProxy -- "3. Backend server returns response through HAProxy" --> WS2
    WS1 -- "4. Response is sent to the client" --> Clients
```

HAProxy Configuration (Example)

```
# Frontend - entry point for client requests
frontend http_front
    bind *:80
    default_backend http_back

# Backend - list of web servers
backend http_back
    balance roundrobin
    server s1 172.16.0.2:80 check
    server s2 172.16.0.3:80 check
```

Web Server Setup (Both Web Servers)

- Install Apache

```
sudo apt install apache2 -y
```
- Edit web page to identify server

```
sudo nano /var/www/html/index.html
```
- Add content (example)

```
<h1>Welcome to Web Server 1</h1>
```

(Use different message for Web Server 2)

Service Management (HAProxy)

```
# Install HAProxy
sudo apt install haproxy -y
# Backup original config
sudo cp /etc/haproxy/haproxy.cfg \
/home/user/haproxy.cfg.bak
# Check configuration
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
# Start and enable service
sudo systemctl restart haproxy
sudo systemctl enable haproxy
sudo systemctl status haproxy
```

Testing from Client

```
# Access HAProxy multiple times
(to test load balancing)
curl http://172.16.0.1
curl http://172.16.0.1
curl http://172.16.0.1
```

You should see responses from both web servers (round-robin).

Failure Testing

- Stop Apache on one backend server

```
sudo systemctl stop apache2
```
- Access HAProxy again (traffic should go to the other server)
- Start Apache again

```
sudo systemctl start apache2
```
- Test again (both servers should work)

Common Issues & Troubleshooting

Problem	Check / Solution
HAProxy not starting	haproxy -c -f /etc/haproxy/haproxy.cfg
Client cannot connect	Check IP, firewall and port 80/443
Only one server receives traffic	Check backend server status and algorithm
Backend server marked down	Check Apache service, port and health check
Wrong response / 404	Check web server configuration
Changes not applied	Restart HAProxy

Useful Check

```
# Check if port 80/443 is in use
sudo ss -lntp | grep -E '':80|:443'

# View HAProxy logs (if enabled)
sudo journalctl -u haproxy -f
```

Summary

HAProxy acts as a reverse proxy and load balancer, providing a single entry point to multiple backend servers. It improves availability, security and performance for web applications.

"Reliable connections. Better performance."

